

**CH2 - Clip 5.**

**학습 데이터 준비 4 (Class  
Imbalance)**

# CONTENT

01

**Class Imbalance란?**

02

**Class Imbalance  
영향성**

03

**Class Imbalance  
다루기**

04

**데이터 접근 방법론  
(Resampling)**

05

**ML 알고리즘  
방법론**

06

**Class Imbalance  
성능 평가 및 선택**

07

**Class  
Imbalance와  
MLOps**

# 1. Class Imbalance 소개

# Class Imbalance 소개

## Class Imbalance란 무엇인가?

- 기계 학습과 분류 문제에서 발생하는 현상 중 하나, 클래스 간의 데이터 불균형을 나타내는 개념
- 하나의 클래스가 다른 클래스에 비해 데이터 포인트 수가 현저히 적을 때 발
- 예) 희귀 질병을 감지하는 분류 모델에서 질병이 있는 경우(양성 클래스) 데이터가 드물게 발생하는 경우

## Class Imbalance가 발생하는 이유?

- 현실 세계의 데이터는 종종 불균형한 데이터 분포를 갖음 -> 대부분의 거래가 정상적인 신용 카드 거래이고, 사기 거래는 드물게 발생
- 데이터 수집 과정에서 클래스 불균형이 발생 -> 특정 클래스에 대한 Sampling 오류, Data 수집 방법, Labeling 오류

# Class Imbalance 소개

## Class Imbalance 중요성

- 실제 ML 응용 분야에서 많이 발생하며, 이를 무시하면 심각한 문제를 초래할 수 있음
- 1) 모델 편향 : 모델이 소수 클래스를 올바르게 식별하지 못할 수 있음
- 2) 비용 고려 : 소수 클래스의 중요도가 높은 경우, 이를 반영하지 않으면 예측의 비용이나 성능이 실제 상황과 맞지 않을 수 있음
- 3) 평가 지표의 왜곡 : 정확도와 같은 평가 지표가 왜곡될 수 있으며, 실제 모델의 성능을 평가하기 어렵게 만듦

## Class Imbalance 해결 필요성

- Class Imbalance 문제를 해결하려면 적절한 기술과 전략을 적용 필요
- 적용을 통해서 모델 성능을 향상시키고, 실제 비즈니스 문제에 더 정확한 예측을 제공할 수 있음

## 2. Class Imbalance 영향성

# Class Imbalance 영향

## Class Imbalance가 모델에 미치는 영향

- 모델 편향 : 소수 클래스를 정확하게 예측하지 못할 가능성이 높음
- 오분류와 비용 : 소수 클래스를 무시하는 경향으로 실제로 큰 문제를 일으킬 수 있음 -> 의료 분야에서 희귀한 질병을 감지하는 모델의 경우, 해당 질병을 놓치는 것은 치명적일 수 있으므로 오분류에 따른 비용이 높을 것임

## 모델 성능 지표에 미치는 영향

- 평가 지표의 왜곡 : Accuracy는 일반적으로 Class Imbalance에 대한 적절한 지표가 아님 -> 모든 클래스를 다수 클래스로 예측하는 "모든 예측이 부정"이라는 간단한 전략으로 높은 정확도를 얻을 수 있음
- Accuracy 외에 precision, recall, f1-score 등의 성능 지표를 고려해야 함
- ROC Curve and AUC
  - ROC Curve : 거짓 양성 비율(FPR)에 대한 진짜 양성 비율(TPR)의 변화를 시각화
  - AUC : ROC 곡선 아래 영역으로, 모델의 성능을 요약하는 지표
- 성능 지표를 신중하게 선택해야 함. 문제의 특성과 비즈니스 요구 사항에 따라 최적의 모델 평가 지표를 설정해야함

## 3. Class Imbalance 다루기

Class Imbalance 조정 목표

Resampling 방법론

ML기반 방법론



# Class Imbalance을 다루는 방법론

## Class Imbalance 조절의 목표

- 소수 클래스에 대한 모델 성능을 향상시키는 것
- 오분류 비용을 줄이고 모델의 실용성을 향상시킬 수 있음

## Class Imbalance 조절 방법의 대분류

- 데이터 기반 접근법 : Resampling 으로 데이터 수를 조절하여 클래스 간 균형을 맞춤
- ML Algorithm 접근법 : model parameter를 조절하여 class imbalance를 고려하도록 모델을 Tuning

# Resampling 기법

## Resampling 개념

- 데이터의 수를 조절하여 Class Imbalance를 다루는 방법
- 클래스간 균형을 맞추거나 OverSampling 혹은 UnderSampling을 통해 클래스를 조절

## Resampling 종류

- OverSampling, UnderSampling, Combined Sampling
- Over Sampling : 소수 클래스 데이터를 증가시키는 방법. 새로운 데이터 포인트를 합성하여 데이터 셋 균형을 맞춤
- Under Sampling : 다수 클래스 데이터를 감소시키는 방법. 다수 클래스에서 일부 데이터를 제거하여 균형을 맞춤

## Resampling 주의사항

- Resampling 기법들의 장단점을 고려해야함.
- OverSampling은 소수 클래스 데이터를 확장하지만, 데이터 중복 및 과적합 문제가 발생할 수 있음
- UnderSampling은 다수 클래스 데이터를 줄이지만 정보 손실이 발생할 수 있음

# 모델 튜닝 및 파라미터 조정

## 모델 튜닝의 필요성

- Class Imbalance를 고려하여 모델을 Tuning하는 과정
- 모델은 소수 클래스를 더 잘 식별할 수 있게 해줌

## Weight 조정

- 가중치를 조절하여, 소수 클래스에 대한 높은 중요도를 부여 가능
- 모델은 소수 클래스를 더 잘 식별할 수 있게됨

## Threshold 조정

- 분류 모델에서 임계값을 조정하여, 모델의 평가 지표를 향상시킬 수 있음
- 예를 들어, recall을 높이기 위해 임계값을 조정할 수 있음

## 4. 데이터 접근 방법론 (Resampling)

Over Sampling

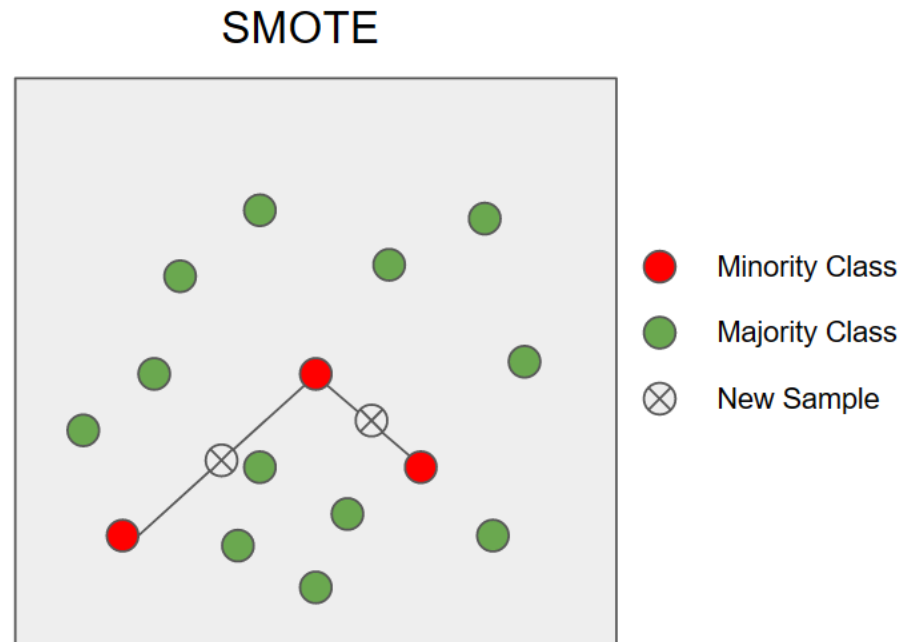
Under Sampling

Combined Sampling

# OverSampling

## SMOTE (Synthetic Minority Over-sampling)

- 소수 클래스 데이터 포인트들을 기존 데이터를 활용하여 데이터를 균형화하는 방법
- 새로운 데이터 포인트는 소수 클래스 데이터 포인트와 그 주변의 이웃 데이터 포인트(랜덤한 데이터를 선택) 사이를 보간하여 생성
- 장점 : 다양한 데이터 생성이 가능하며 모델의 일반화 능력을 향상시킬 수 있음
- 단점 : 데이터 중복이 발생할 수 있음

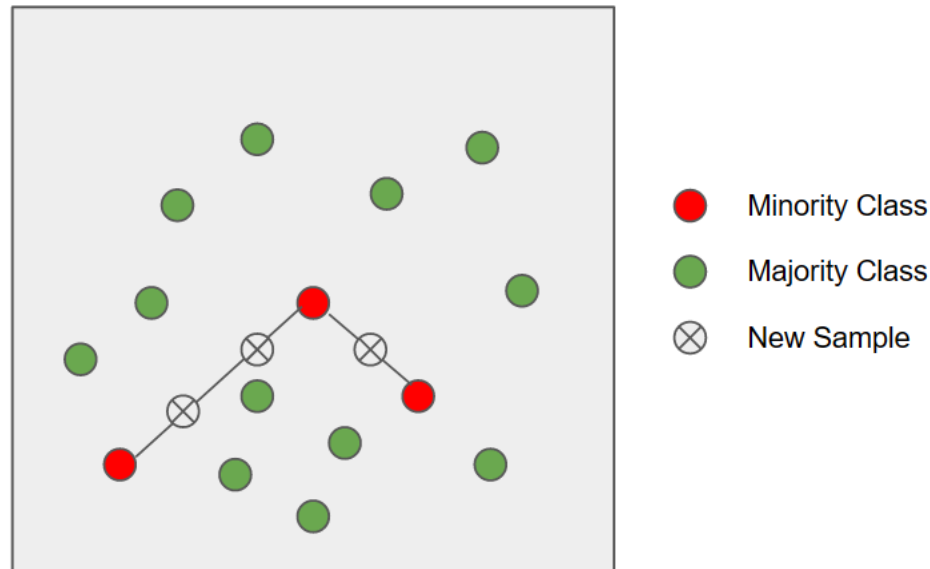


# OverSampling

## ADASYN (Adaptive Synthetic Sampling)

- 소수 클래스 데이터 포인트들을 기존 데이터를 활용하여 데이터를 균형화하는 방법
- 소수 클래스 데이터 포인트의 가중치를 계산하고, 높은 가중치를 가지는 데이터 포인트에 대해 더 많은 합성을 수행
- 가중치는 데이터 분포와 클래스 간 거리에 따라 동적으로 조정. 거리가 가까운 데이터 포인트에 대한 합성이 더욱 강조됨
- 장점 : 데이터 분포(거리)에 더욱 더 적응적이며, 클래스 간 거리에 따라 합성을 조절하여 불균형을 더 효과적으로 다룰 수 있음
- 단점 : SMOTE에 비해 더 복잡하며 계산 비용을 높일 수 있음

ADASYN



# OverSampling

## SMOTE vs ADASYN

| SMOTE                           | 비교 사항             | ADASYN                               |
|---------------------------------|-------------------|--------------------------------------|
| 가중치를 고려하지 않고, 고정 비율에 따라 랜덤하게 합성 | Neary by 데이터 선택방법 | 가중치를 고려하여 동적으로 합성                    |
| 미리 정의된 비율에 따라 합성 데이터 양이 조절      | 합성 데이터 양의 조절      | 데이터 분포와 클래스 간 거리에 따라 합성 데이터 양을 자동 조절 |
| 높은 비용이 발생                       | 복잡성               | 비교적 간단하게 구현 가능                       |
| 데이터 특성에 따라 다른                   | 성능 차이             | 데이터 특성에 따라 다른                        |

# UnderSampling

## Random UnderSampling

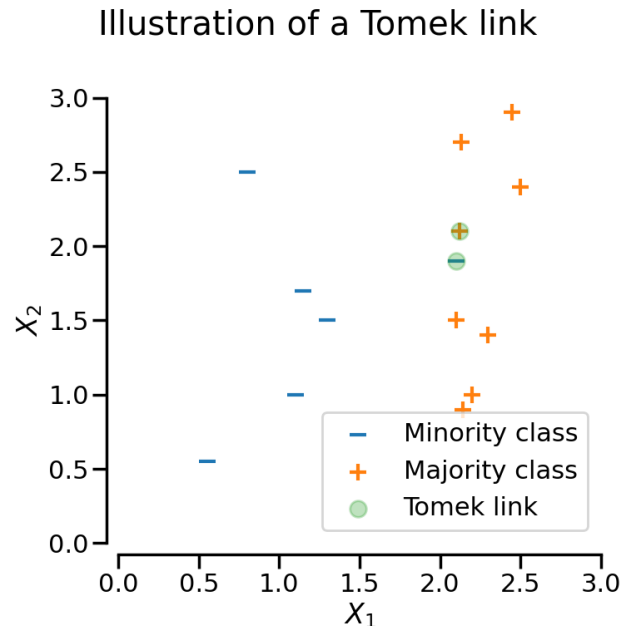
- 랜덤 언더샘플링은 다수 클래스 데이터 포인트 중에서 무작위로 일부 데이터를 선택하여 소수 클래스와의 균형을 맞춤
- 장점 : 간단하고 빠르게 구현할 수 있음
- 단점 : 정보 손실이 발생할 수 있으며, 선택된 데이터가 소수 클래스를 잘 대표하지 못할 수 있음



# UnderSampling

## Tomek Links UnderSampling

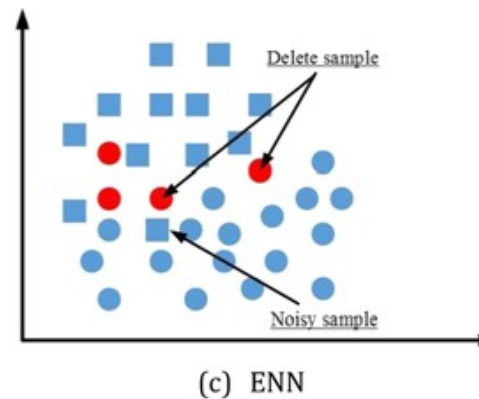
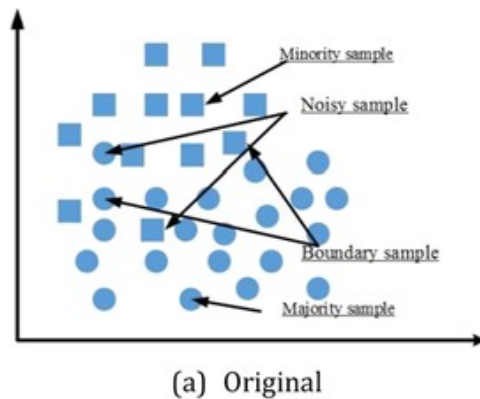
- 클래스 간 거리가 가까운 데이터 포인트 쌍 중에서 다수 클래스의 데이터를 제거하는 방법
- 수행 과정.
  - 1) 다수 클래스와 소수 클래스 사이의 모든 데이터 간의 거리를 계산
  - 2) Tomek Links 식별 : 서로 다른 클래스에 속한 데이터 포인트 쌍 중에서 클래스 간 거리가 가장 가까운 데이터 포인트 쌍을 의미
  - 3) Tomek Links 제거 : 식별된 Tomek Links를 데이터에서 제거



# UnderSampling

## ENN (Edited Nearest Neighbors)

- 클래스 간 거리가 가까운 데이터 포인트 쌍 중에서 다수 클래스의 데이터를 제거하는 방법
- 수행 과정
  - 1) 다수 클래스 데이터 포인트에 대해, 해당 데이터 포인트와 가장 가까운 k개의 이웃을 찾음
  - 2) 다수 클래스 데이터 포인트 중에서 소수 클래스로 잘못 분류된 데이터 포인트를 식별
  - 3) 이런 오분류된 데이터 포인트를 제거하여 다수 클래스 데이터를 정제
  - 4) 모델이 잘못 분류하는 오분류 데이터를 제거하고 모델의 성능을 향상



# UnderSampling

## TomekLinks와 ENN 공통점

- 클래스 간 거리를 고려하여 데이터 포인트를 수정하며, Class Imbalance 문제를 해결해서 모델 성능을 향상시키는 공동 목표 가짐

## TomekLinks와 ENN 차이점

- Tomek Links는 클래스 간 거리가 가까운 데이터 포인트 중에서 '서로 다른 클래스'에 속한 데이터 쌍을 식별하여 제거
- ENN은 각 다수 클래스 데이터 포인트에 대해 가장 가까운 이웃을 기반으로 오분류된 데이터(소수 데이터로 판별되는 것)를 찾아내서 제거
- Tomek Links는 클래스 간 경계를 명확하게 만드는 것이 목적
- ENN은 다수 클래스 데이터를 정제하고 오분류를 줄이는 데 목적

# UnderSampling

## TomekLinks와 ENN 공통점

- 클래스 간 거리를 고려하여 데이터 포인트를 수정하며, Class Imbalance 문제를 해결해서 모델 성능을 향상시키는 공동 목표 가짐

## TomekLinks와 ENN 차이점

| TomekLinks                                  | 비교 사항  | ENN                                                      |
|---------------------------------------------|--------|----------------------------------------------------------|
| 클래스 간 거리가 가까운 데이터 포인트 중에서 '서로 다른 클래스' 쌍을 식별 | 제거 데이터 | 각 다수 클래스 데이터 포인트에 대해 그룹을 형성하고 가장 가까운 이웃을 기반으로 오분류 데이터 식별 |
| 클래스간 경계를 명확하게                               | 구현 목적  | 다수 클래스 데이터를 정제하고 오분류를 줄이는 데 목적                           |

# Combined Sampling

## SMOTEENN

- SMOTE와 ENN 두 기술을 함께 사용하여 클래스 불균형을 해결
- 동작 방식
  - 1) SMOTE를 사용한 오버샘플링
  - 2) ENN을 사용한 언더샘플링
  - 3) 균형된 데이터셋 구성
- 장점 : 두 기술을 조합함으로써 클래스 불균형을 효과적으로 해결 가능
- 단점 : 모델 성능은 데이터셋의 특성과 문제에 따라 정보 손실이 발생할 수 있으며, 실험과 비교 분석이 필요

## 5. ML알고리즘 기반 방법론

# ML 알고리즘 기반 방법론

## ML 알고리즘 방법론

- 모델 tuning 및 parameter 조정은 머신러닝 모델의 성능을 최적화하고 Class Imbalance 데이터에 대응하기 위한 중요한 전략 중 하나
- 모델 알고리즘 선택, 클래스 가중치 조정, 임계값 조정, 적절한 Metric 사용, Hyper-parameter Tuning, Validation, Ensemble

# ML 알고리즘 방법론

## Model Algorithm 선택

- Class Imbalance에 Sensitive한 모델 알고리즘을 선택
- Binary Classification 에서는 SVM, Random Forest, Gradient Boosting과 같은 모델이 좋은 성능을 보임

## Class Weight 조정

- 모델의 Loss Function에 class weight를 부여하여 imbalance class에 높은 가중치(weight)을 할당
- 모델은 imbalance dataset을 더 중요하게 취급하여 학습 진행

## Threshold 조정

- 모델의 예측 Threshold를 조정하여 Precision과 Recall을 조절 할 수 있음

## 적절한 Metric 사용

- Accruacy가 일반적으로 Imbalance Class에서 부적절하므로, 다른 Metric을 기반으로 모델을 최적화



# ML 알고리즘 방법론

## Hyper-Parameter Tuning

- 모델의 Hyper Parameter를 조정하여 최적의 모델을 찾음

## Validation

- Cross Validation을 통해 모델의 일반화 능력을 평가하고, Class Imbalance에 대해서도 성능 신뢰성을 확인 필요

## Ensemble 활용

- 다양한 모델을 경합하는 앙상블 기법을 사용하여 Class Imbalance 문제를 다룸

## 6. Class Imbalance 성능 평가 및 선택

평가 중요성

Precision, Recall, F1 Score

RoC, AUC

# 모델 성능을 어떻게 평가할 것인가?

## 성능 평가의 중요성

- 모델의 성능을 평가하는 것은 머신러닝 프로젝트에서 매우 중요한 부분
- 모델이 얼마나 잘 작동하고 예측을 얼마나 신뢰할 수 있는지를 판단

## 평가 Metric 선택

- 모델의 목적과 특성에 따라 다양한 성능 Metric을 선택할 수 있으며, 일반적으로 사용되는 Metric으로는 Accuracy, Precision, Recall, F1Score등이 있다.

# Class Imbalance에서 Precision, Recall, F1-Score 중요성

## Precision (정밀도)

- 모델이 예측한 양성 클래스 중에서 실제로 양성인 샘플의 비율
- 거짓 양성(FP)를 줄이고 모델이 정확한 예측을 하는 데 얼마나 잘 하는지를 나타냄

## Recall (재현율)

- 실제 양성 클래스 중에서 모델이 양성으로 예측한 샘플의 비율
- 거짓 음성(FN)을 줄이고 모델이 실제 양성을 놓치지 않는 데 얼마나 잘 하는지를 나타냄

모델이 Major class로  
모두 예측하는 경우에도  
영향을 받지 않는 Metric

## F1-Score

- Precision과 Recall의 조화 평균
- 비즈니스 목적에서 Precision과 Recall 모두 중요한 경우 활용

# Class Imbalance에서 ROC Curve과 AUC

## ROC Curve

- 다양한 Threshold에서 모델의 Recall 대비 거짓 양성 비율을 나타내는 그래프
- 모델의 성능을 시각적으로 평가하며, Threshold 선택의 중요성을 강조

## AUC

- AUC는 ROC 곡선 아래 영역을 나타내며, 모델의 전반적인 성능을 단일 숫자로 나타냄
- 모델 성능을 요약할 때 유용하며, 0.5(무작위 예측) ~ 1(완벽한 예측) 사이의 값을 가짐

**모델이 Major class로  
모두 예측하는 경우에도  
영향을 받지 않는 Metric**

# Class Imbalance Metric

## 예시

- 의료 진단 모델 : 의료 분야에서는 Precision이 매우 중요하며, Recall도 중요한 경우가 많음. 특히 의료 진단 모델은 Class Imbalance가 심하므로 주의해야함
- 스팸 메일 필터링 : Precision과 Recall을 균형있게 유지하는 것이 중요하므로, F1-Score를 평가에 사용할 수 있음
- Class Imbalance 모델 비교 : ROC Curve 혹은 AUC를 활용하여 모델 전반적인 성능을 비교

## 7. Class Imbalance와 MLOps

통합 필요 이유

MLOps 구현시 고려사항

# MLOps에 Class Imbalance 통합

## 통합이 필요한 이유

- Class Imbalance는 모델의 정확도 및 신뢰성에 영향을 미침
- 실제 운영 환경에서 모델의 성능을 보장하는 것이 중요하기 때문에, 이에 영향을 미치는 Class Imbalance 관리 필요

## 통합 방법

- **데이터 수집 및 전처리 단계** : Class Imbalance를 고려하여 데이터 수집 및 전처리 과정에서 적절한 Resampling 기법을 적용
- **모델 훈련 단계** : 모델 훈련 과정에서 가중치 조정 및 성능 지표 설정을 통해 Class Imbalance를 고려
- **평가 및 모니터링** : 모델의 실시간 평가 및 모니터링에서 모델이 불균형한 Input에도 정상적으로 동작하는지 확인 필요



# MLOps에 Class Imbalance 통합

## Model Update에서의 Class Imbalance 관리

- **모델의 시간적 변화** : 실제 환경에서 데이터 및 클래스 분포가 시간이 지남에 따라 변함. 이로 인해 새롭게 발생하는 Class Imbalance 문제가 발생할 수 있으므로, 모델을 update하고 재학습해야함.
- **모델 Update와 Re- training 전략** : 정기적인 모델 update 및 실시간 데이터 스트림에서 모델을 re training (fine tuining)하는 방법을 고려해야 함
- **Data Drift 및 Class Imbalance** : 데이터 분포의 변경(Data Drift)는 Class Imbalance와 관련성이 있을 수 있으므로 고려 필요
- **모델 버전 관리** : 모델 성능 개선과 비교를 위해 필요
- **모델 성능 모니터링** : Class Imbalance가 성능에 어떻게 영향을 미치는지 추적 모니터링
- **자동화된 Update 및 Re-Training** : MLOps에 자동화된 프로세스를 도입하여, Class Imbalance 관련 문제가 감지되면 모델의 자동 update 및 re-training을 수행하는 방법을 개발 적용